

©2023 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works. DOI: 10.1109/LRA.2023.3315209

Probably Approximately Correct Nonlinear Model Predictive Control (PAC-NMPC)

Adam Polevoy^{1,2}, Marin Kobilarov², and Joseph Moore^{1,2}

Abstract—Approaches for stochastic nonlinear model predictive control (SNMPC) typically make restrictive assumptions about the system dynamics and rely on approximations to characterize the evolution of the underlying uncertainty distributions. For this reason, they are often unable to capture more complex distributions (e.g., non-Gaussian or multi-modal) and cannot provide accurate guarantees of performance. In this paper, we present a sampling-based SNMPC approach that leverages recently derived sample complexity bounds to certify the performance of a feedback policy without making assumptions about the system dynamics or underlying uncertainty distributions. By parallelizing our approach, we are able to demonstrate real-time receding-horizon SNMPC with statistical safety guarantees in simulation and on hardware using a 1/10th scale rally car and a 24-inch wingspan fixed-wing UAV.

Index Terms—Planning under Uncertainty, Integrated Planning and Control, Robot Safety

I. INTRODUCTION

NONLINEAR model predictive control (NMPC) has proven to be a powerful approach for controlling high-dimensional, complex robotic systems (e.g., [1], [2], [3], [4]). Nevertheless, although these methods can handle large state spaces, nonlinear dynamics, and system constraints, their performance can be adversely affected by the presence of uncertainty even in the context of real-time replanning [5]. A number of approaches have been proposed to compensate for this marginal robustness, the simplest of which is to generate a feedback policy to track the current receding-horizon plan (e.g., [2]). These approaches, however, do not account for uncertainty or closed-loop performance during the planning process. To address this, approaches such as robust NMPC (RNMP) and stochastic NMPC (SNMPC) attempt to reason about the response of the policy to disturbances during planning. This is usually accomplished by making simplifying assumptions about the dynamics and disturbances. For instance, in RNMP, disturbance sets may be approximated by ellipsoids [6], or robustness guarantees may require the existence of a control contraction metric (CCM) [7]; in

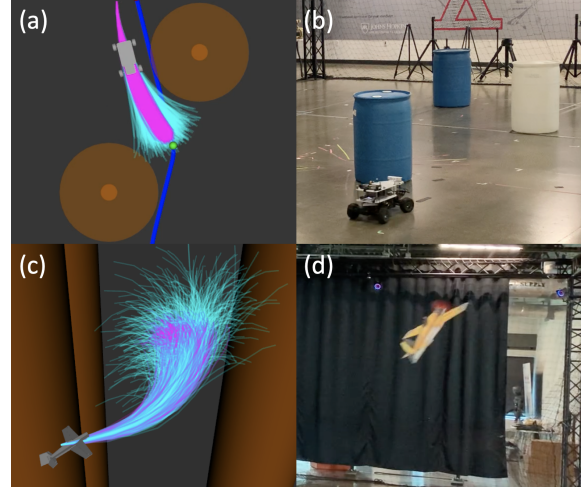


Fig. 1. PAC-NMPC optimizes high confidence bounds on the expected cost/constraint of feedback policy distributions. We demonstrated our approach on a 1/10th-scale Rally Car (a, b) and an Edge 540 fixed-wing UAV (c, d).

SNMPC, uncertainty distributions are often approximated via Gaussians [8] or sampling [9].

In this paper, we present a sampling-based SNMPC algorithm capable of controlling stochastic dynamical systems without applying limiting assumptions to the structure of the stochastic dynamics or underlying uncertainty distributions. In addition, our approach leverages recently derived sample complexity bounds [10] to provide a probabilistic guarantee on system performance. Our algorithm builds on Probably Approximately Correct Robust Policy Search (PROPS) [11] to directly optimize an upper confidence bound on the expected cost and the probability of constraint violation for a receding-horizon feedback policy. Not only does this approach encourage robust policy generation, but the minimized bounds themselves provide a statistical guarantee for each planning interval. To achieve real-time performance, we use a graphics processing unit (GPU) to parallelize our algorithm. We then evaluate our approach in simulation and on hardware using a 1/10th scale rally car and a 24-inch wingspan fixed-wing UAV. Our contributions are:

- 1) A novel algorithm, PAC-NMPC, for receding horizon SNMPC with probabilistic performance guarantees.
- 2) A real-time implementation via GPU acceleration.
- 3) Demonstration of our algorithm on complex underactuated systems via simulation and hardware experiments.

II. RELATED WORK

Prior research addresses the impact of uncertainty on NMPC via robust NMPC (RNMP) or stochastic NMPC (SNMPC).

Manuscript received: May, 7, 2023; Revised August, 1, 2023; Accepted August, 24, 2023.

This paper was recommended for publication by Editor Clement Gosselin upon evaluation of the Associate Editor and Reviewers' comments. This work was supported by JHU APL internal R&D

¹Adam Polevoy and Joseph Moore are with the Applied Physics Lab, Johns Hopkins University, Laurel, MD 20723 USA adam.polevoy@jhuapl.edu, joseph.moore@jhuapl.edu

²Marin Kobilarov is with the Whiting School of Engineering, Department of Mechanical Engineering, Johns Hopkins University, Baltimore, MD 21218 mkobilar1@jhu.edu

Digital Object Identifier (DOI): see top of this page.

RNMPC represents uncertainty in the system as bounded disturbances. While min-max tube NMPC methods [6], [12], [13] and multi-stage NMPC approaches [14] have been proposed, constraint-tightening based tube NMPC [15] has proved to be the most common and computationally tractable RNMPC approach. Most of these approaches rely on tube size derived from a computed Lipschitz constant [16], [17], CCMs [7], [18], [19], or bounds on incremental stability [20]. These methods for computing invariant sets are often complex, may place significant restrictions on the dynamics, and can lead to overly conservative results. Recent work has proposed sampling-based reachability analysis for arbitrary continuously differentiable dynamics with less conservative results [21], [22].

Stochastic NMPC represents uncertainty as a probability distribution and optimizes the expectation over costs and constraint violations. Oftentimes, they optimize a feedback policy, rather than a control sequence. Our method, PAC-NMPC, falls into this category of approaches. Early approaches focused on a local iterative solution to the stochastic Hamilton-Jacobi-Bellman equation [23], [24], which relies on a first order approximation of the dynamics for noise propagation. Stochastic differential dynamic programming approaches [25] would later allow for second order approximation of the dynamics. More recent methods utilize the unscented transform to propagate noise through nonlinear dynamics [8], [26]. In [27], the authors leverage path integral control to generate open-loop trajectories and create a sampling-based NMPC algorithm. More recent extensions [9], [28] include a feedback policy for robust performance, [29] provides guarantees on free-energy growth, and [30] penalizes Conditional Value-at-Risk (CVaR).

Sampling-based stochastic optimal control methods are closely related to policy search in reinforcement learning. A general background and survey of common policy search methods can be found in [31]. In [11], researchers developed an approach for robust policy search using a Probably Approximately Correct (PAC) learning framework called PAC Robust Policy Search (PROPS) based on recently derived sample complexity bounds in [10]. Researchers have also recently explored (PAC)-Bayes theory to generate collision-avoidance policies with performance guarantees in novel environments [32] and achieve vision-based planning with motion primitives [33]. Our PAC-NMPC method builds on the work in [10] and [11] to develop a highly generalizable sampling-based stochastic feedback motion-planning algorithm that can provide statistical performance guarantees for a large class of uncertain dynamical systems.

III. BACKGROUND

Before presenting our approach for Probably Approximately Correct NMPC, we first review the recently derived PAC bounds for Iterative Stochastic Policy Optimization which are the foundation for our approach.

A. Iterative Stochastic Policy Optimization

Consider the stochastic dynamics given by $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ defined by a vector of state values $\mathbf{x}_t \in \mathbb{R}^{N_x}$, a vector of

Algorithm 1: Episodic ISPO

```

1 Inputs:  $\hat{\nu}_0^*$ ,  $i \leftarrow 0$ ;
2 while termination condition not met do
3   for  $j = 1, \dots, M$  do
4     Sample Trajectory:  $(\tau_j, \xi_j) \sim p(\cdot, \cdot | \hat{\nu}_i^*)$ ;
5     Evaluate Cost:  $J_j = J(\tau_j)$ ;
6    $\hat{\nu}_{i+1}^* = \arg \min_{\nu} \hat{\mathcal{J}}(\nu) + \alpha D(\nu, \hat{\nu}_i^*)$ ;
7    $i \leftarrow i + 1$ 
8 return  $\hat{\nu}_i^*$ 

```

control inputs $\mathbf{u}_t \in \mathbb{R}^{N_u}$, and probability density p . Iterative Stochastic Policy Optimization (ISPO) [10] formulates the search for a control policy $\mathbf{u}_t = \pi_t(\mathbf{x}_t, \xi)$ as a stochastic optimization problem, where ξ is a set of policy parameters. Specifically, ISPO introduces a surrogate distribution $p(\xi|\nu)$ where policy parameters ξ are dependent on distribution hyper-parameters ν . This induces a joint distribution

$$p(\tau, \xi|\nu) = p(\tau|\xi)p(\xi|\nu) \quad (1)$$

where $p(\tau|\xi)$ represents the natural stochasticity of the system and is given as $p(\tau|\xi) = p(\mathbf{x}_0) \prod_{t=0}^T p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$, where $\mathbf{u}_t = \pi(\mathbf{x}_t, \xi)$. Here τ represents the discrete time trajectory sequence $\{\mathbf{x}_0, \mathbf{u}_0, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_{N_T}, \mathbf{x}_{N_T+1}\}$, where N_T is the number of timesteps.

In general, the objective of ISPO is to solve the optimization problem $\nu^* = \arg \min_{\nu} \mathcal{J}(\nu)$ where $\mathcal{J}(\nu) = \mathbb{E}_{\tau, \xi \sim p(\cdot, \cdot | \nu)}[J(\tau)]$. To do this, ISPO iteratively samples from Eq. 1 and solves the optimization problem $\hat{\nu}_{i+1}^* = \arg \min_{\nu} \hat{\mathcal{J}}(\nu) + \alpha D(\nu, \hat{\nu}_i^*)$ for each iteration i until convergence. Here $\hat{\mathcal{J}}(\nu)$ is an empirical approximation of the expected cost, D is a distance between distributions, often the $\mathcal{KL}$ -divergence, and $\alpha > 0$ is hand-tuned weight or Lagrange multiplier [34] computed for a constraint on $D(\nu, \hat{\nu}_i^*)$. Alg. 1 is a general framework for ISPO.

B. PAC Bounds for Stochastic Policy Search

Instead of directly minimizing an empirical approximation of the expected cost, it can be beneficial to minimize an *upper confidence bound* on the expected cost. Not only does such an approach encourage robust policies, but it also can provide guarantees on future performance. In this paper, we directly optimize the PAC bounds derived in [10]. While these bounds are derived in [10] and discussed in [11], we review them here since they are fundamental to our approach.

In [10] the presented PAC bound $\mathcal{J}_{\alpha}^+(\nu)$ takes the form

$$\mathcal{J}_{\alpha}^+(\nu) \triangleq \hat{\mathcal{J}}_{\alpha}(\nu) + \alpha d(\nu) + \Phi_{\alpha}(\delta) \quad (2)$$

where $\hat{\mathcal{J}}_{\alpha}(\nu)$ is a robust estimator [35] of the expected cost, $d(\nu)$ is a distance between distributions, and $\Phi_{\alpha}(\delta)$ is a concentration-of-measure term which accounts for discrepancies between the true mean, $\mathcal{J}(\nu)$, and the robust estimator, $\hat{\mathcal{J}}_{\alpha}(\nu)$. For a particular choice of $\hat{\mathcal{J}}_{\alpha}(\nu)$, $d(\nu)$, and $\Phi_{\alpha}(\delta)$, [10] proves that $\mathcal{J}_{\alpha}^+(\nu)$ bounds the expected cost $\mathcal{J}(\nu)$ with a probability of $1 - \delta$.

To derive an expression for $\hat{\mathcal{J}}_{\alpha}(\nu)$, we must estimate

$$\mathcal{J}(\nu) = \mathbb{E}_{\tau, \xi \sim p(\cdot, \cdot | \nu_0)} \left[J(\tau) \frac{p(\xi|\nu)}{p(\xi|\nu_0)} \right]. \quad (3)$$

Because the likelihood ratio $\frac{p(\xi|\nu)}{p(\xi|\nu_0)}$ can be unbounded, a robust estimation technique is needed. Following [35], the expected value of a random variable X can be approximated as $\mathbb{E}[X] \approx \frac{1}{\alpha M} \sum_{i=0}^M \psi(\alpha X_i)$ for some $\alpha > 0$, where X_i is a sampled value of X and $\psi(x) = \log(1 + x + \frac{1}{2}x^2)$. Given L prior surrogate distributions with hyper-parameters $\nu_0, \nu_1, \dots, \nu_{L-1}$ and M iid samples from each joint distribution, $(\tau_{i0}, \xi_{i0}), (\tau_{i1}, \xi_{i1}), \dots, (\tau_{iM}, \xi_{iM}) \sim p(\cdot, \cdot | \nu_i)$, the robust estimate of the expected cost can then be written as

$$\hat{\mathcal{J}}_\alpha(\nu) \triangleq \frac{1}{\alpha LM} \sum_{i=0}^{L-1} \sum_{j=1}^M \psi(\alpha \ell_{ij}) \quad (4)$$

$$\ell_{ij} = J(\tau_{ij}) \frac{p(\xi_{ij}|\nu)}{p(\xi_{ij}|\nu_i)} \quad (5)$$

To compute the distance between the distributions, the Renyi divergence, D_2 , is used. $d(\nu)$ is given as

$$d(\nu) \triangleq \frac{1}{2L} \sum_{i=0}^{L-1} b_i^2 e^{D_2(p(\cdot|\nu) || (p(\cdot|\nu_i)))} \quad (6)$$

$$0 \leq J(\tau_{ij}) \leq b_i \quad \forall j = 0, \dots, M \quad (7)$$

The concentration-of-measure term $\Phi_\alpha(\delta)$ is given as

$$\Phi_\alpha(\delta) \triangleq \frac{1}{\alpha LM} \log \frac{1}{\delta}. \quad (8)$$

It tightens the bound as the number of samples increases and as the bound confidence, $1 - \delta$, decreases.

In addition to cost-functions, we can also incorporate the probability of constraint violation, $\mathcal{C}(\nu)$, of the form $\mathcal{C}(\nu) = \mathbb{P}(g(\tau) > 0) = \mathbb{E}_{\tau, \xi \sim p(\cdot, \cdot | \nu)} [\mathbb{I}_{\{g(\tau) > 0\}}]$, where $\mathbb{I}$ is the indicator function. Similarly, we can derive $\mathcal{C}_\alpha^+(\nu)$ which upper bounds $\mathcal{C}(\nu)$ with probability $1 - \delta$.

To combine costs and constraints, we optimize a weighted sum of $\mathcal{J}_\alpha^+(\nu)$ and $\mathcal{C}_\alpha^+(\nu)$, where $\gamma > 0$ is a heuristically selected weighting coefficient. We solve

$$\nu_{i+1} = \nu^* = \arg \min_{\nu} \min_{\alpha > 0} (\mathcal{J}_\alpha^+(\nu) + \gamma \mathcal{C}_\alpha^+(\nu)) \quad (9)$$

using a GPU implementation of L-BFGS-B [36]. When optimizing the bounds, we used self-normalized importance weights to compute ℓ_{ij} in Eq. 5 and analytical gradients, as described in [11].

IV. APPROACH

To formulate a receding-horizon NMPC algorithm that leverages the PAC bounds in [10], we first address the stochastic trajectory optimization problem. We then extend this approach to enable real-time feedback motion planning.

A. PAC Stochastic Trajectory Optimization

To formulate stochastic trajectory optimization using PAC bounds, we consider a parameterization of a discrete-time open-loop policy $\mathbf{u}_t = \pi_t(\xi) = \zeta_t$, where $\xi = [\zeta_0^T \ \zeta_1^T \ \dots \ \zeta_{N_T}^T]^T$, $\zeta_t \in \mathbb{R}^{N_u}$, and N_T is the time horizon for the trajectory.

We parameterize the surrogate distribution $p(\xi|\nu)$ as a multivariate Gaussian over this discrete-time control trajectory so that $\xi \sim \mathcal{N}(\xi|\mu, \Sigma)$. For computational efficiency,

Algorithm 2: PAC Stochastic Trajectory Optimization

```

1 Inputs:  $\hat{\nu}_0^*, \mathbf{x}_0, i \leftarrow 0$ ;
2 while termination condition not met do
3   for  $j = 1, \dots, M$  do // GPU Parallelized
4      $\tau_{ij}, \xi_{ij} \sim p(\cdot, \cdot | \nu_i, \mathbf{x}_0)$ ;
5      $J_{ij} = J(\tau_{ij}), c_{ij} = c(\tau_{ij})$ ;
6    $\hat{\nu}_{i+1}^* \leftarrow \arg \min_{\nu} \min_{\alpha > 0} (\mathcal{J}_\alpha^+(\nu) + \gamma \mathcal{C}_\alpha^+(\nu))$ ;
7    $i \leftarrow i + 1$ ;
8 return  $\hat{\nu}_i^*, \mathcal{J}_\alpha^+(\hat{\nu}_i^*), \mathcal{C}_\alpha^+(\hat{\nu}_i^*)$ 
```

Algorithm 3: $\tau \sim p(\cdot | \xi, \mathbf{x}_0)$

```

1 Inputs:  $\xi = [\mathbf{u}_0^T, \dots, \mathbf{u}_{N_T}^T]^T, \mathbf{x}_0$ ;
2 for  $t = 0, \dots, N_T$  do // Stochastic Rollout
3    $\mathbf{x}_{t+1} \sim p(\cdot | \mathbf{x}_t, \mathbf{u}_t)$ ;
4 Return  $\tau = \{\mathbf{x}_0, \mathbf{u}_0, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_{N_T}, \mathbf{x}_{N_T+1}\}$ 
```

we assume a diagonal covariance matrix Σ so the distribution parameters are given as $\nu \triangleq [\mu^T, \text{diag}(\Sigma)^T]^T$ where $\text{diag}(\Sigma) = [\eta_0^T \ \eta_1^T \ \dots \ \eta_{N_T}^T]^T$ and $\eta_t \in \mathbb{R}^{N_u}$. Thus, a control trajectory with an N_u -dimensional control input and N_T timesteps, the surrogate distribution is parameterized as a $N_u N_T$ -dimensional multivariate Gaussian where $\mu \in \mathbb{R}^{N_u N_T}$ and $\text{diag}(\Sigma) \in \mathbb{R}^{N_u N_T}$.

For each optimization iteration, M trajectories are sampled from the joint distribution (Eq. 1) and evaluated using the augmented cost in Eq. 9. Sampling from the joint distribution is achieved by first sampling policy parameters from the surrogate distribution and then using these policy parameters to sample from the stochastic dynamics as shown in Alg. 3. The augmented costs of the samples are used to find a new set of policy parameters that minimize the weighted sum of the cost and constraint bounds (Alg. 2). This process iterates until a termination condition (e.g. maximum time, convergence metric, etc.) is reached. The optimization not only returns the optimized policy parameters, but also the optimized bounds themselves.

An analytical formulation for the surrogate distribution is necessary to compute the robust estimates (Eq. 4) and Renyi divergences (Eq. 7). By contrast, the only requirement on the stochastic dynamics is to permit sampling. This property allows our algorithm to generalize to a large class of complex (potentially “black-box”) stochastic dynamics models. In practice, the cost and constraint functions can also be stochastic (e.g. $J_{ij}, c_{ij} \sim p(\cdot, \cdot | \tau_{ij})$).

B. PAC Feedback Motion Planning

While the ability to compute PAC open-loop policies is valuable, ideally, we would like to compute closed-loop policies of the form $\pi_t(\mathbf{x}_t, \xi)$. A closed-loop policy should enable us to generate tighter state-space trajectory distributions and improve the ability of our policies to satisfy control objectives. However, designing feedback policies is not straightforward and can be very system specific. One fairly general approach might be to select a local feedback control policy of the form $\mathbf{u}_t = \mathbf{K}_t(\mathbf{x}_t^d - \mathbf{x}_t) + \mathbf{u}_t^d$, where $\kappa = \{\mathbf{K}_0, \mathbf{K}_1, \dots, \mathbf{K}_{N_T}\}$ is a sequence of time-varying gains, $\mathbf{K}_t \in \mathbb{R}^{N_u \times N_x}$, $\mathbf{x}_t^d \in \mathbb{R}^{N_x}$ is a nominal state trajectory and $\mathbf{u}_t^d \in \mathbb{R}^{N_u}$ is a nominal input tra-

Algorithm 4: $\tau \sim p(\cdot|\xi, \mathbf{x}_0)$ with Feedback

```

1 Inputs:  $\xi = [\mathbf{u}_0, \dots, \mathbf{u}_{N_T}], \mathbf{x}_0$ ;
2  $\mathbf{x}_0^d = \mathbf{x}_0$ ;
3 for  $k = 0, \dots, N_T$  do // Nominal Rollout
4    $\mathbf{x}_{t+1}^d = \mathbf{x}_t^d + f(\mathbf{x}_t^d, \mathbf{u}_t^d)\Delta t$ ;
5  $\tau^d = \{\mathbf{x}_0^d, \mathbf{u}_0^d, \mathbf{x}_1^d, \mathbf{u}_1^d, \dots, \mathbf{u}_{N_T}^d, \mathbf{x}_{N_T+1}^d\}$ ;
6  $\kappa = \text{TVLQR}(\tau^d)$ ;
7 for  $t = 0, \dots, N_T$  do // Feedback Rollout
8    $\mathbf{u}_t = \mathbf{K}_t(\mathbf{x}_t - \mathbf{x}_t^d) + \mathbf{u}_t^d$ ;
9    $\mathbf{x}_{t+1} \sim p(\cdot|\mathbf{x}_t, \mathbf{u}_t)$ ;
10 Return  $\tau = \{\mathbf{x}_0, \mathbf{u}_0, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_{N_T}, \mathbf{x}_{N_T+1}\}$ 

```

jectory. All of the elements of these sequences are parameters of the feedback policy. Assuming the surrogate distribution is a multivariate Gaussian with a diagonal covariance matrix, in this case, $\xi \in \mathbb{R}^{2(N_x N_u + N_x + N_u)N_T}$. Given the size of the decision space, this parameterization of the policy is unlikely to be computationally tractable. To overcome this challenge, we instead use a local closed-loop policy of the form

$$\mathbf{u}_t = \mathbf{K}_t(\tau^d(\xi, \mathbf{x}_0))(\mathbf{x}_t^d(\xi, \mathbf{x}_0) - \mathbf{x}_t) + \mathbf{u}_t^d(\xi). \quad (10)$$

Here we refer to $\tau^d \triangleq \{\mathbf{x}_0^d, \mathbf{u}_0^d, \mathbf{x}_1^d, \mathbf{u}_1^d, \dots, \mathbf{u}_{N_T}^d, \mathbf{x}_{N_T+1}^d\}$ as the nominal trajectory, which is computed using a nominal deterministic discrete-time dynamics model, $\mathbf{f}(\mathbf{x}_t^d, \mathbf{u}_t^d)$,

$$\mathbf{x}_{t+1}^d = \mathbf{x}_t^d + \mathbf{f}(\mathbf{x}_t^d, \mathbf{u}_t^d)\Delta t \quad (11)$$

where $\mathbf{u}_t^d = \zeta_t$. To sample from this feedback policy, we must first compute the nominal trajectory, τ^d . However, τ^d itself is dependent on ξ ; for this reason, we employ a two-stage sampling process. In the first stage, we sample from the trajectory distribution $p(\tau^d|\xi)$ using the deterministic dynamics model (Eq. 11) and the surrogate distribution $p(\xi|\nu)$. For each sample trajectory τ^d , we compute the time-varying gains $\mathbf{K}_t(\tau^d)$. In the second stage, we sample from the closed-loop stochastic dynamics using the feedback policy (Eq. 10). This approach, summarized in Alg. 4, certifies the feedback policy by allowing our sampled costs and constraints to capture the impact of our local feedback policy on our stochastic dynamics model.

We use the finite horizon, discrete, time-varying linear quadratic regulator (TVLQR) [37] to compute the feedback gains $\mathbf{K}_t(\tau^d)$. For each sampled nominal trajectory, the gains are computed by integrating the finite horizon discrete-time Riccati equations backward in time using the nominal dynamics linearized about τ^d .

C. PAC-NMPC

Finally, we propose a receding-horizon NMPC approach based on our PAC feedback motion planning algorithm (Alg. 5). At each planning interval, with a period of H , the trajectory distribution and corresponding feedback policy is optimized given the current state, $\mathbf{x}_0$, and the prior hyper-parameters, $\hat{\nu}_0^*$. Then, the trajectory and feedback gains are trimmed by $\frac{H}{\Delta t}$ to account for time passed since the beginning of the planning interval and executed by the controller. The trim operation is given as $\text{Trim}(\kappa) = \{\mathbf{K}_{\frac{H}{\Delta t}}, \dots, \mathbf{K}_{N_T}\}$. Hyper-parameters determining the first $\frac{H}{\Delta t}$ time steps of the control trajectory

Algorithm 5: PAC-NMPC

```

1 Input:  $\hat{\nu}_0^*$ ;
2  $\mathbf{x}_0 = \text{GetCurrentState}()$ ;
3 while objective not completed do
4    $\hat{\nu}^* = \text{Optimize}(\hat{\nu}_0^*, \mathbf{x}_0)$ ; // Alg. 2
5    $\mathbf{u}^d = \text{MaximumLikelihoodEstimate}(\hat{\nu}^*)$ ;
6   for  $t = 0, \dots, N_T$  do // Nominal Rollout
7      $\mathbf{x}_{t+1}^d = \mathbf{x}_t^d + f(\mathbf{x}_t^d, \mathbf{u}_t^d)\Delta t$ ;
8      $\tau^d = \{\mathbf{x}_0^d, \mathbf{u}_0^d, \mathbf{x}_1^d, \mathbf{u}_1^d, \dots, \mathbf{u}_{N_T}^d, \mathbf{x}_{N_T+1}^d\}$ ;
9      $\kappa = \text{TVLQR}(\tau^d)$ ;
10     $\tau^d, \kappa, \hat{\nu}^* \leftarrow \text{Trim}(\tau^d, \kappa, \hat{\nu}^*)$ ;
11    Execute  $(\tau^d, \kappa)$ ;
12     $\mathbf{x}_0 = \text{GetCurrentState}()$ ;
13     $\hat{\nu}_0^* = \text{InitializePrior}(\hat{\nu}^*, \tau^d, \kappa, \mathbf{x}_0)$ ; // Alg. 6
14 Return

```

Algorithm 6: Initialize Prior

```

1 Input:  $\hat{\nu}^*, \tau^d, \kappa, \mathbf{x}_0$ ;
2  $[\mu_0^T, \mu_1^T, \dots, \mu_{N_T - \frac{H}{\Delta t}}^T, \eta_0^T, \eta_1^T, \dots, \eta_{N_T - \frac{H}{\Delta t}}^T]^T = \hat{\nu}^*$ ;
3  $\mathbf{u}^d = \mu$ ;
4 for  $t = 0, \dots, N_T - \frac{H}{\Delta t}$  do // Feedback Rollout
5    $\mathbf{u}_t = \mathbf{K}_t(\mathbf{x}_t - \mathbf{x}_t^d) + \mathbf{u}_t^d$ ;
6    $\mathbf{x}_{t+1} = \mathbf{x}_t + f(\mathbf{x}_t, \mathbf{u}_t)\Delta t$ ;
7 for  $t = N_T - \frac{H}{\Delta t} + 1, \dots, N_T$  do // Extend prior
8    $\mathbf{u}_t = \mathbf{0}$ ;
9    $\eta_t = \max(\eta_{N_T - \frac{H}{\Delta t}}, \eta_{\min})$ ;
10 Return  $\hat{\nu}_0^* = [\mathbf{u}_0^T, \mathbf{u}_1^T, \dots, \mathbf{u}_{N_T}^T, \eta_0^T, \eta_1^T, \dots, \eta_{N_T}^T]^T$ 

```

represent control signals that the system is executing during the optimization and are not modified during the optimization. To execute the policy, we use the maximum likelihood estimate of the policy parameters, which is simply the mean, μ , of the multivariate Gaussian $p(\xi|\nu)$.

Given that optimal trajectory distributions are expected to be similar between subsequent planning intervals, we use the optimal hyper-parameters from the last planning interval to initialize the prior policy distribution for the next. This initialization is extremely important since it allows the prior to start near an optimal solution, thus reducing the required number of iterations for convergence. After trimming, we apply the feedback policy to generate a modified policy parameter distribution with a mean feasible for the current initial state. Then, we extend the mean with zeros and the variance with its final value. We threshold the prior variance with $\eta_{\min}$ to prevent it from starting too small, which would inhibit exploration (Alg. 6).

V. SIMULATION EXPERIMENTS

A. Trajectory Optimization Experiments

We simulate a stochastic bicycle model with acceleration and steering rate inputs. We denote $\mathbf{x}_t = [x_0, x_1, x_2, x_3, x_4]^T$ as the state vector, $\mathbf{u}_t = [u_0, u_1]^T$ as the control vector, $l = 0.33$ as the wheel base, and $\Gamma = \text{diag}([0.001, 0.001, 0.1, 0.2, 0.001])$ as the covariance.

$$\begin{aligned}
\mathbf{x}_{t+1} &\sim p(\cdot|\mathbf{x}_t, \mathbf{u}_t) \triangleq \mathbf{x}_t + (f(\mathbf{x}_t, \mathbf{u}_t) + \omega)\Delta t \\
f(\mathbf{x}_t, \mathbf{u}_t) &= [x_3 \cos(x_2), x_3 \sin(x_2), x_3 \tan(x_4)/l, u_0, u_1] \\
\omega &\sim \mathcal{N}(\cdot|\mathbf{0}, \Gamma)
\end{aligned} \quad (12)$$

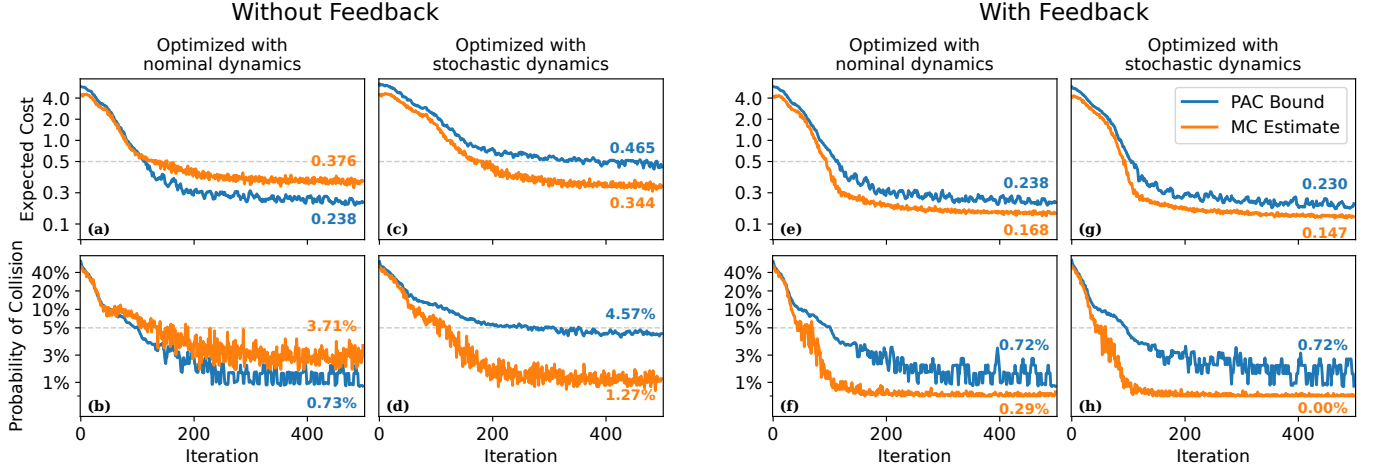


Fig. 2. Convergence of PAC Bound compared to MC estimates (evaluated with stochastic dynamics). Y-axis switches from log to linear scale halfway

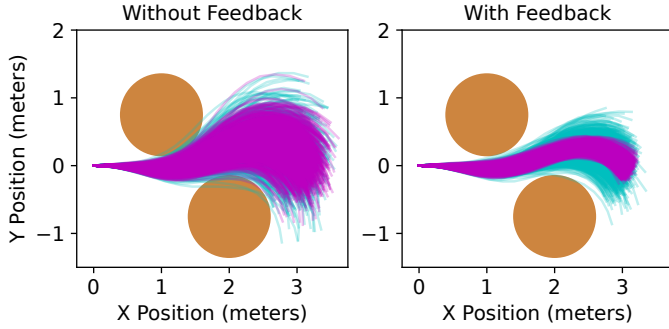


Fig. 3. Samples from optimized policy distribution and stochastic dynamics in cyan. Samples from mean of the optimized policy distribution and stochastic dynamics in magenta. Obstacles in brown.

A 20 timestep trajectory with $\Delta t = 0.1$ sec is optimized, with an initial state of $\mathbf{x}_I = [0.0, 0.0, 0.0, 1.0, 0.0]^T$ and a goal state of $\mathbf{x}_G = [3.0, 0.0, 0.0, 1.0, 0.0]^T$. The steering angle is constrained to $(-0.4, 0.4)$ rad, the steering rate to $(-1.0, 1.0)$ $\frac{\text{rad}}{\text{sec}}$, and the acceleration to $(-1.0, 1.0)$ $\frac{\text{m}}{\text{s}^2}$. We apply a quadratic cost on the final state of the trajectory: $J(\tau) = \mathbf{x}_{N_T+1}^T \mathbf{Q}_f \mathbf{x}_{N_T+1}$ where $\mathbf{Q}_f = \text{diag}([2.0, 2.0, 0.0, 0.0, 0.0])$. An obstacle constraint is applied with circular obstacles at $(1.0, 0.75)$ and $(2.0, -0.75)$. The initial prior distribution has a mean of all zeros and a variance of all ones. Parameters were as follows: $L = 5$, $M = 1024$, $\delta = 0.05$, $\gamma = 10$. Simulations were run on a laptop with an Intel Core i9-9880H CPU and a Nvidia GeForce RTX 2070 GPU.

We ran trajectory optimization for 500 iterations. Once completed, our method returns the hyper-parameters, $\hat{\nu}^*$, and the optimized PAC bounds, $\mathcal{J}_\alpha^+(\hat{\nu}^*)$ and $\mathcal{C}_\alpha^+(\hat{\nu}^*)$. It is guaranteed that policies sampled from $p(\xi|\nu^*)$ will have an expected cost of $\leq \mathcal{J}_\alpha^+(\hat{\nu}^*)$ and an expected constraint (probability of collision) of $\leq \mathcal{C}_\alpha^+(\hat{\nu}^*)$ with a 95% chance. To validate the PAC bounds, we compare them against Monte Carlo estimates of the expected cost, $\hat{\mathcal{J}}(\hat{\nu}^*)$, and probability of collision, $\hat{\mathcal{C}}(\hat{\nu}^*)$, at each iteration.

To highlight the necessity of properly considering stochasticity to generate accurate guarantees, we compared performance while optimizing the PAC bounds with and without considering stochastic dynamics (replace line 3 of Alg. 3 and

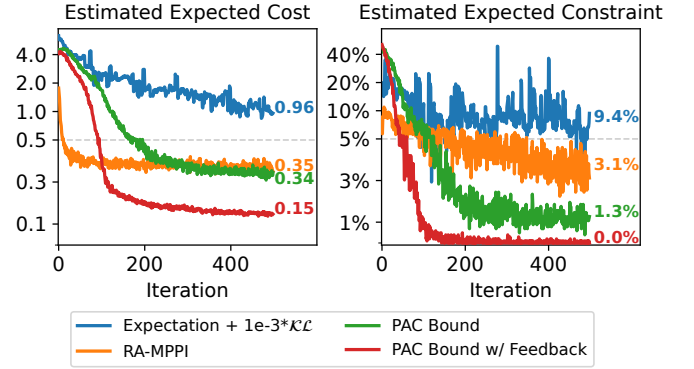


Fig. 4. Comparison between our approach and other methods.

line 9 of Alg. 4 with $\mathbf{x}_{t+1} = \mathbf{x}_t + f(\mathbf{x}_t, \mathbf{u}_t)\Delta t$). Optimizing without stochastic dynamics caused $\hat{\mathcal{J}}(\hat{\nu}^*)$ and $\hat{\mathcal{C}}(\hat{\nu}^*)$ to be larger than the bounds, indicating that the guarantees were not accurate (Fig. 2a,b). We also performed an ablation study in which we compared performance with and without feedback. On average, each iteration took 15.9 ms without feedback and 18.5 ms with feedback. Feedback enabled the optimizer to reduce the bounds to lower values (Fig. 2e-h) and resulted in a tighter distribution in state-space (Fig 3).

We compare against optimizing an empirical approximation of the expected costs/constraints, regulated by the $\mathcal{KL}$ -divergence to prior policies, to demonstrate that optimizing the PAC bounds encourages more robust policies. We also compare against RA-MPPI [30], a state-of-the-art risk-aware sampling based NMPC method. We used the following parameters: $\Sigma_\epsilon = 0.01$, $M = 1024$, $N = 300$, $\eta = 0$, $\alpha = 0.3$, $A = 10$, $B = 1$, $C_u = 0.5$ for the cost, and $C_u = 0.05$ for the constraint. Our approach converged to the lowest estimated expected cost and probability of collision (Fig 4).

B. NMPC Experiment

The system follows a receding horizon state generated on a looping path. The path runs along a set of circular obstacles. We optimize a 12 timestep trajectory with $\Delta t = 0.1$ second at a replanning period of $H = 0.2$ sec. We apply a quadratic cost

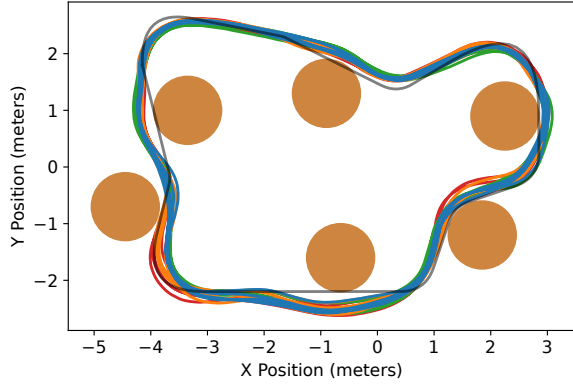


Fig. 5. Paths taken while following route (black) and avoiding obstacles (brown) in simulation. Blue/green paths use stochastic bounds, red/orange paths do not. Blue/orange paths use feedback, red/green paths do not.

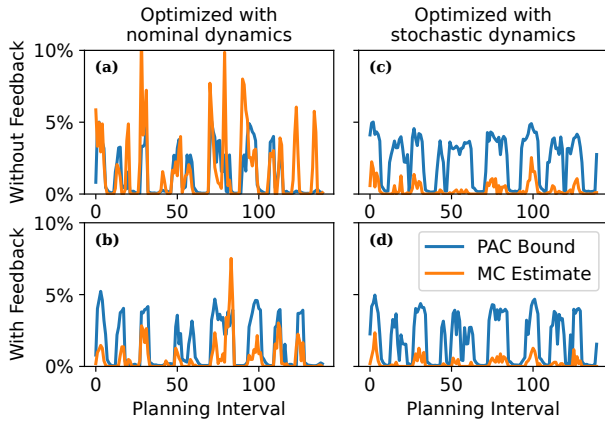


Fig. 6. Optimized PAC Bound compared to Monte Carlo estimates for probability of constraint violation at each planning interval during first laps. Variations in the PAC Bound and MC estimate over time are expected behavior due to the robot moving closer/farther from obstacles along the route.

on the final state of the trajectory: $J(\tau) = \mathbf{x}_{N_T+1}^T \mathbf{Q}_f \mathbf{x}_{N_T+1}$ where $\mathbf{Q}_f = \text{diag}([1.0, 1.0, 0.1, 0.1, 0.0])$. Parameters were as follows: $L = 5$, $M = 1024$, $\delta = 0.05$, $\gamma = 10$. To allow for a higher control rate, the executed control trajectory is interpolated from 10 Hz to 50 Hz with the assumption that our PAC bounds will still hold. We perform an ablation study in which we compare performance with and without feedback, as well as optimizing the PAC bounds with and without stochastic dynamics.

PAC-NMPC was successfully able to control the system in real time and followed the path while avoiding nearby obstacles. In Figure 5, we show the route taken by the system during five loops of the path.

During each planning interval of $H = 0.2$ sec, the controller achieved an average of 54 iterations of PAC stochastic trajectory optimization (approximately 3.7ms per iteration). The controller consistently produced PAC bounds $\leq 5\%$ probability of collision, which accurately bounded the Monte Carlo estimates when optimizing with stochastic dynamics. The bound was exceeded only once out of 482 planning intervals. Thus, the estimates were bounded accurately in around 99.8% of planning intervals, well within the 95% confidence bound (Fig 6d).

We demonstrate a quantitative difference in the observance

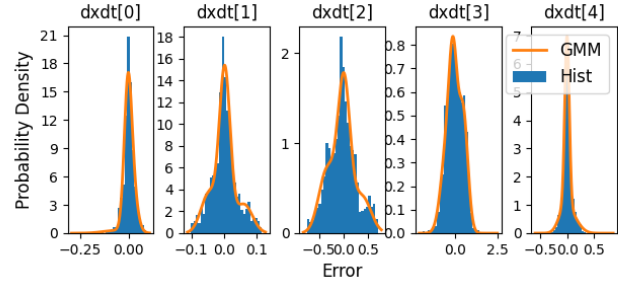


Fig. 7. Rallycar dynamics noise model

of the PAC Bounds in Figure 6. When optimizing the bounds with the nominal dynamics, the probability of collision estimates were only bounded in 93.9% of planning intervals (Fig 6b). This decrease in bound accuracy is even more apparent when running the controller without feedback to compensate for unmodeled noise, resulting in only 64.7% of planning intervals having bounded probability of collision estimates (Fig 6a).

VI. HARDWARE EXPERIMENTS

A. Rally Car

1) *Hardware*: To evaluate our method on physical hardware, we ran PAC-NMPC on a 1/10th-scale Traxxas Rally Car platform. The algorithm runs on a Nvidia Jetson Orin mounted on the bottom platform. The control interface is a variable electronic speed controller (VESC), which executes commands and provides servo state information. The position and orientation is tracked with an OptiTrack system.

2) *Setup*: The route, virtual obstacle placements, and TVLQR costs are identical to those in the simulation experiments. We optimize a 12 timestep trajectory with $\Delta t = 0.1$ sec at a replanning period of $H = 0.2$ sec. We apply a quadratic cost to the final state of the trajectory: $J(\tau) = \mathbf{x}_{N_T+1}^T \mathbf{Q}_f \mathbf{x}_{N_T+1}$ where $\mathbf{Q}_f = \text{diag}([1.0, 1.0, 0.4, 0.1, 0.0])$. Parameters were $L = 2$, $M = 1024$, $\delta = 0.05$, $\gamma = 10$.

We model the rally car dynamics as a stochastic bicycle model where noise is modeled as a 3 component Gaussian Mixture Model. This mixture model was fit from 2500 samples of data collected with the car (Fig. 7). We perform the same ablation study as done in the simulation experiments.

3) *Results*: PAC-NMPC ran onboard the rally car in real-time and followed the path while avoiding nearby obstacles. In Figure 8, we display the path taken by the car with collisions annotated by arrows. During each planning interval of $H = 0.2$ sec, the controller achieved an average of 10 iterations of trajectory distribution optimization with approximately 20ms per iteration.

The controller consistently produced PAC bounds $\leq 10\%$ probability of collision, which accurately bounded the Monte Carlo estimates of the probability of collision at all planning intervals (Fig. 9d). In this context, Monte Carlo estimates refer to sampled trajectories using the simulated stochastic dynamics model. When optimizing with stochastic dynamics and without feedback, the car collided with virtual obstacles twice, likely due to unmodeled dynamics not captured in the Gaussian mixture model (Fig. 9c). When optimizing the bound

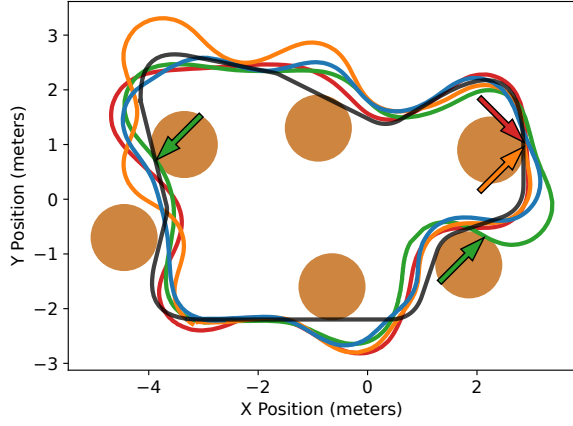


Fig. 8. Path taken while following route (black) and avoiding obstacles (brown) with rally car hardware. Blue/green paths use stochastic bounds, red/orange paths do not. Blue/orange paths use feedback, red/green paths do not. Collisions are annotated by arrows.

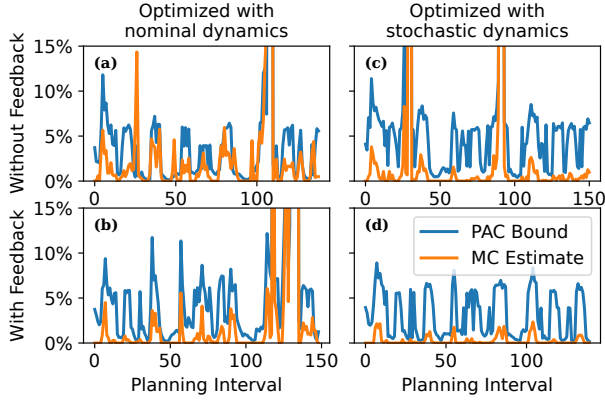


Fig. 9. Optimized PAC Bound compared to Monte Carlo estimates for probability of constraint violation at each planning interval.

with nominal dynamics, the car collided with virtual obstacles, with and without feedback (Fig. 9a,b).

B. Fixed-Wing UAV

1) *Hardware*: To demonstrate that our approach can extend to more complex, high-dimensional systems, we use PAC-NMPC to control a 24" wingspan Edge 540 fixed-wing UAV. It has a controllable propeller, rudder, elevator, and kinematically linked ailerons. The position and orientation are tracked with an OptiTrack system.

2) *Setup*: We utilize a quaternion formulation of the fixed-wing described in [2] with RK2 integration. This system has a 17-dimensional state space and a 4-dimensional control space. The state of the system is $\mathbf{x} = [\mathbf{r}, \mathbf{q}, \boldsymbol{\delta}, \mathbf{v}, \boldsymbol{\omega}, p]$ where $\mathbf{r} \in \mathbb{R}^3$ is the position, $\mathbf{q} \in \mathbb{Q}$ is the orientation, $\boldsymbol{\delta} \in \mathbb{R}^3$ are control surface deflections of the ailerons, elevator, and rudder respectively, $\mathbf{v} \in \mathbb{R}^3$ is the linear velocity, $\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity, and p is the propeller speed. The control signal is the rate of change of the control surface deflections and of the propeller speed: $\mathbf{u} = [u_a, u_e, u_r, u_p]$. We place noise represented by a 3 component Gaussian Mixture Model over the body frame accelerations, which was fit from 2000 samples of data collected with the fixed-wing (Fig. 10). We optimize a 12 timestep trajectory with $\Delta t = 0.1$ sec at a replanning

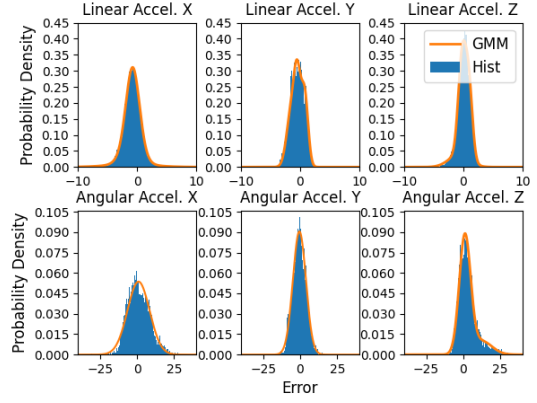


Fig. 10. Edge540 Acceleration Noise Model

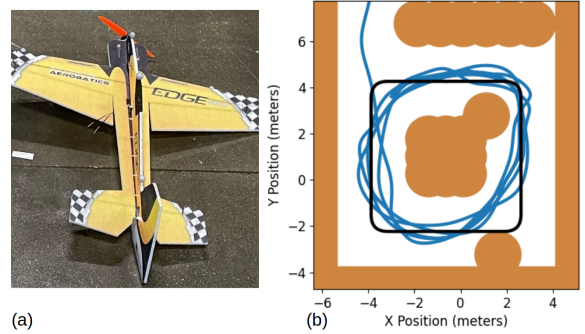


Fig. 11. a) Edge540 b) Path (blue) taken by the fixed-wing while following route (black) and avoiding obstacles (brown).

period of $H = 0.2$. We apply a quadratic cost of the form $J(\tau) = \mathbf{x}_{N_T+1}^T \mathbf{Q}_f \mathbf{x}_{N_T+1} + \sum_{t=1}^{N_T} (\mathbf{x}_t^T \mathbf{Q} \mathbf{x}_t + \mathbf{u}_t^T \mathbf{R} \mathbf{u}_t)$ and an obstacle constraint. Parameters were as follows: $L = 1$, $M = 1024$, $\delta = 0.05$, and $\gamma = 10$. We use feedback and use stochastic dynamics to optimize the PAC bounds.

To enable real-time performance, we set the number of prior policies, L , to one and compute the linearized dynamics for the feedback policy using finite difference on the mean trajectory of the prior. The controller was run on a desktop computer with an AMD Ryzen 7 5800x and a NVIDIA GeForce RTX 3080.

3) *Results*: PAC-NMPC was successfully able to control the fixed-wing in real-time, following a square path while avoiding nearby obstacles. In Figure 11, we display the route taken by the system during five loops of the path. During each planning interval of $H = 0.2$ sec, the controller achieved an average of 22 iterations of trajectory distribution optimization with around 8.5 ms per iteration. The controller consistently produced PAC bounds of around 10% probability of collision, which accurately bounded the Monte Carlo estimates (Fig. 12).

VII. DISCUSSION

In this work, we presented a novel SNMPC method capable of propagating uncertainty through arbitrary nonlinear dynamic systems and of providing statistical guarantees on expected cost and constraint violations. We demonstrated real time performance both in simulation and on-board physical hardware. Further, we show that the algorithm is capable

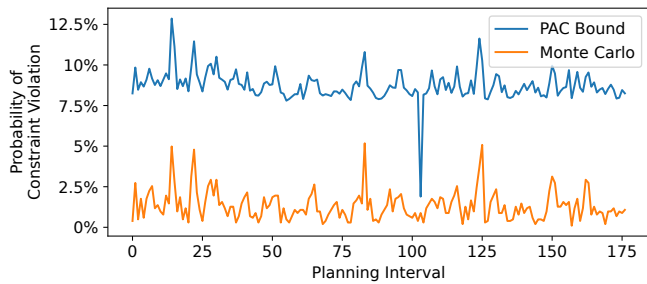


Fig. 12. Optimized PAC Bound compared to Monte Carlo estimates for probability of constraint violation.

of scaling to more complex systems, like fixed-wing UAVs. Since this algorithm can be used with “black-box” sampling of dynamics (assuming they are continuously differentiable), costs, constraints, and noise, future work can investigate its use with learned dynamics and perception-informed costs. Future work could also explore the extension of this approach to more complex control trajectory distributions.

REFERENCES

- [1] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, “Pampc: Perception-aware model predictive control for quadrotors,” in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1–8.
- [2] M. Basescu and J. Moore, “Direct nmpe for post-stall motion planning with fixed-wing uavs,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 9592–9598.
- [3] Y. Ding, A. Pandala, and H.-W. Park, “Real-time model predictive control for versatile dynamic motions in quadrupedal robots,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8484–8490.
- [4] R. Grandia, F. Farshidian, R. Ranftl, and M. Hutter, “Feedback mpc for torque-controlled legged robots,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019, pp. 4730–4737.
- [5] J. A. Paulson and A. Mesbah, “An efficient method for stochastic optimal control with joint chance constraints for nonlinear systems,” *International Journal of Robust and Nonlinear Control*, vol. 29, no. 15, pp. 5017–5037, 2019.
- [6] Z. Manchester and S. Kuindersma, “Dirtrel: Robust trajectory optimization with ellipsoidal disturbances and lqr feedback,” in *Robotics: Science and Systems*. Cambridge, MA, USA, 2017.
- [7] S. Singh, A. Majumdar, J.-J. Slotine, and M. Pavone, “Robust online motion planning via contraction theory and convex optimization,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 5883–5890.
- [8] N. Ozaki, S. Campagnola, and R. Funase, “Tube stochastic optimal control for nonlinear constrained trajectory optimization problems,” *Journal of Guidance, Control, and Dynamics*, 2020.
- [9] J. Yin, Z. Zhang, E. Theodorou, and P. Tsotras, “Trajectory distribution control for model predictive path integral control using covariance steering,” in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 1478–1484.
- [10] M. Kobilarov, “Sample complexity bounds for iterative stochastic policy optimization,” *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [11] M. Sheckells, “Probably approximately correct robust policy search with applications to mobile robotics,” Ph.D. dissertation, Johns Hopkins University, 2020.
- [12] M. E. Villanueva, R. Quirynen, M. Diehl, B. Chachuat, and B. Houska, “Robust mpc via min–max differential inequalities,” *Automatica*, vol. 77, pp. 311–321, 2017.
- [13] G. Garimella, M. Sheckells, J. L. Moore, and M. Kobilarov, “Robust obstacle avoidance using tube nmpe,” in *Robotics: Science and Systems*, 2018.
- [14] S. Lucia, R. Paulen, and S. Engell, “Multi-stage nonlinear model predictive control with verified robust constraint satisfaction,” in *53rd IEEE Conference on Decision and Control*. IEEE, 2014, pp. 2816–2821.
- [15] D. Q. Mayne, E. C. Kerrigan, E. Van Wyk, and P. Falugi, “Tube-based robust nonlinear model predictive control,” *International journal of robust and nonlinear control*, vol. 21, no. 11, pp. 1341–1353, 2011.
- [16] D. L. Marruedo, T. Alamo, and E. F. Camacho, “Input-to-state stable mpc for constrained discrete-time nonlinear systems with bounded additive uncertainties,” in *Proceedings of the 41st IEEE Conference on Decision and Control*, 2002., vol. 4. IEEE, 2002, pp. 4619–4624.
- [17] Y. Gao, A. Gray, H. E. Tseng, and F. Borrelli, “A tube-based robust nonlinear predictive control approach to semiautonomous ground vehicles,” *Vehicle System Dynamics*, vol. 52, no. 6, pp. 802–823, 2014.
- [18] F. Bayer, M. Bürger, and F. Allgöwer, “Discrete-time incremental iss: A framework for robust nmpe,” in *2013 European Control Conference (ECC)*. IEEE, 2013, pp. 2068–2073.
- [19] G. Chou, N. Ozay, and D. Berenson, “Model error propagation via learned contraction metrics for safe feedback motion planning of unknown systems,” in *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, 2021, pp. 3576–3583.
- [20] J. Köhler, R. Soloperto, M. A. Müller, and F. Allgöwer, “A computationally efficient robust model predictive control framework for uncertain nonlinear systems,” *IEEE Transactions on Automatic Control*, vol. 66, no. 2, pp. 794–801, 2020.
- [21] T. Lew and M. Pavone, “Sampling-based reachability analysis: A random set theory approach with adversarial sampling,” in *Conference on robot learning*. PMLR, 2021, pp. 2055–2070.
- [22] A. Wu, T. Lew, K. Solovey, E. Schmerling, and M. Pavone, “Robust-rrt: Probabilistically-complete motion planning for uncertain nonlinear systems,” in *Robotics Research*. Springer, 2023, pp. 538–554.
- [23] E. Todorov and W. Li, “A generalized iterative lqg method for locally-optimal feedback control of constrained nonlinear stochastic systems,” in *Proceedings of the 2005, American Control Conference, 2005*. IEEE, 2005, pp. 300–306.
- [24] E. Todorov and Y. Tassa, “Iterative local dynamic programming,” in *2009 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning*. IEEE, 2009, pp. 90–95.
- [25] E. Theodorou, Y. Tassa, and E. Todorov, “Stochastic differential dynamic programming,” in *Proceedings of the 2010 American Control Conference*. IEEE, 2010, pp. 1125–1132.
- [26] T. A. Howell, C. Fu, and Z. Manchester, “Direct policy optimization using deterministic sampling and collocation,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5324–5331, 2021.
- [27] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Aggressive driving with model predictive path integral control,” in *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 1433–1440.
- [28] G. Williams, B. Goldfain, P. Drews, K. Saigol, J. M. Rehg, and E. A. Theodorou, “Robust sampling based model predictive control with sparse objective information,” in *Robotics: Science and Systems*, 2018.
- [29] M. S. Gandhi, B. Vlahov, J. Gibson, G. Williams, and E. A. Theodorou, “Robust model predictive path integral control: Analysis and performance guarantees,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1423–1430, 2021.
- [30] J. Yin, Z. Zhang, and P. Tsotras, “Risk-aware model predictive path integral control using conditional value-at-risk,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 7937–7943.
- [31] M. P. Deisenroth, G. Neumann, J. Peters *et al.*, “A survey on policy search for robotics,” *Foundations and Trends® in Robotics*, vol. 2, no. 1–2, pp. 1–142, 2013.
- [32] A. Majumdar, A. Farid, and A. Sonar, “Pac-bayes control: learning policies that provably generalize to novel environments,” *The International Journal of Robotics Research*, vol. 40, no. 2–3, pp. 574–593, 2021.
- [33] S. Veer and A. Majumdar, “Probably approximately correct vision-based planning using motion primitives,” in *Conference on Robot Learning*. PMLR, 2021, pp. 1001–1014.
- [34] J. Peters, K. Mulling, and Y. Altun, “Relative entropy policy search,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, no. 1, 2010, pp. 1607–1612.
- [35] O. Catoni, “Challenging the empirical mean and empirical variance: a deviation study,” in *Annales de l’IHP Probabilités et statistiques*, vol. 48, no. 4, 2012, pp. 1148–1185.
- [36] Y. Fei, G. Rong, B. Wang, and W. Wang, “Parallel l-bfgs-b algorithm on gpu,” *Computers & graphics*, vol. 40, pp. 1–9, 2014.
- [37] R. Tedrake, *Underactuated Robotics*, 2022. [Online]. Available: <http://underactuated.mit.edu>